

程序设计实践

The Practice of Programming

课程简介

课程目标

《程序设计实践》课程主要介绍一些程序设计的实践方法，面向的是已经先修了程序设计语言，算法，数据结构等基础课程之后的学生。

《程序设计实践》课程从软件开发的完整过程的角度审视程序设计中需要注意的方方面面的问题，了解如何写出不但运行正确，而且可读性好，设计优雅，接口清晰，运行健壮，跨平台，高性能的程序。大语言模型辅助的编程技术也应用于本课程中，以帮助学生了解人工智能时代下软件编程的新范式。

学生在《程序设计实践》课程作业中能够亲身实践课程讲授的内容，从而理解这些方法的意义，让这些方法能够对以后的毕业设计，研究生阶段的科研工作，或者毕业后的工作岗位上从事的编程工作有所帮助。

教材和参考书目

- 程序设计实践简明教程，王玉龙 编著，北京邮电大学出版社，2024版
- 程序设计实践(英文版)， [美] 布莱恩 W.克尼汉（Brian W. Kernighan）， [美] 罗勃·派克（Rob Pike） 著，人民邮电出版社，2016版
- Beyond Vibe Coding - From Coder to AI-Era Developer, Addy Osmani 著，O'REILLY出版社，2025版

Beyond Vibe Coding

1. 引言：什么是Vibe Coding（基本概念，工具介绍，优势，局限性）
2. 提示的艺术：与AI有效沟通（提示工程的概念，提示的技巧，提示进阶）
3. 70%问题：真正有效的AI辅助工作流（AI能帮你完成 70% 的工作，但最后那 30% 令人沮丧）
4. 超越70%问题：最大化人类的贡献（拥抱 AI 擅长的前70%的事情，提升自己对剩余30%所需的技能和洞察力）
5. 理解生成的代码：审查、优化、拥有（作为开发者，你不能只是拿走AI的输出就盲目地部署它。你需要审查它、测试它、可能还需要改进它，并将它与代码库的其余部分集成）
6. AI驱动的原型设计：工具与技术（如何利用AI生成原型，如何将原型向生产过渡）
7. 使用AI构建一个WEB应用
8. 安全性，可维护性，可靠性（AI生成的代码在安全性，可维护性，可靠性上将会有诸多问题，如何应对）
9. Vibe Coding的伦理问题（AI生成的代码可能不符合本地政治文化习惯，存在版权问题。与大模型交互带来的隐私问题等等。如何解决？）
10. 更自主的辅助编程智能体（新技术的发展带来更加自主的智能体，它们的风险，如何应对？）
11. 超越代码生成：AI辅助开发的未来（未来的AI辅助开发，不仅仅是代码生成，而是在分析，设计，调试，维护等各方面提供支持，它带来的挑战是什么，如何应对？）

学习方法

- **课堂讲授**
任课老师课上讲解相关知识，占20学时
- **课下自学**
学生以课上知识为基础，通过老师推荐的参考资料和学习工具，查阅资料和动手实践，从而加深理解知识点和扩展知识面
- **实验**
学生利用上机时间完成实验作业，占12学时
- **线上答疑**
老师通过电子邮件或者微信群回答学生提问

课程内容

设计与实现

风格

接口

排错

测试

性能

可移植性

记法

以大作业为例，讲述一个项目的需求分析，设计，实现的过程，数据结构和算法的选择，以及相关的技术介绍。

课程内容

设计与实现

风格

接口

排错

测试

性能

可移植性

记法

从命名，表达式，语句，段落，宏定义，常量，注释等几个方面，讨论什么样的风格会让程序的可读性易维护性更好，更加不容易出错。

课程内容

设计与实现

风格

接口

排错

测试

性能

可移植性

记法

接口分为程序与程序接口以及人与程序接口两个方面。接口为什么重要？如何设计良好的接口？

课程内容

设计与实现

风格

接口

排错

测试

性能

可移植性

记法

程序的哪些地方最容易出错？

排错的工具，方法？

排错的基本流程是什么？

课程内容

设计与实现

风格

接口

排错

测试

性能

可移植性

记法

如何设计测试驱动和测试桩？

如何设计自动测试脚本？

如何进行压力测试？

如何进行性能测试？

课程内容

设计与实现

风格

接口

排错

测试

性能

可移植性

记法

影响性能的主要原因是什么？

如何寻找性能瓶颈？

如何优化提高性能？

性能分析的工具，方法。

课程内容

设计与实现

风格

接口

排错

测试

性能

可移植性

记法

当一个程序面对不同的服务器厂商，不同的操作系统，不同的编译器，不同语言的客户，应该注意哪些问题？

课程内容

设计与实现

风格

接口

排错

测试

性能

可移植性

记法

记法为什么很重要？我们应该
如何看待已有的记法？必要的
时候是否应该设计自己的记法？
记法解释执行的原理？

课程安排

共**16**周， **16**次课

10周， 课堂讲授

6周， 程序设计实验

考核方式

本课程的考核采取平时成绩+大作业的方式。

成绩组成：

✓ 平时成绩

- 通过小实验作业考核
- 占成绩30%

✓ 大作业

- 代码审查、文档检查、功能演示、答辩
- 占成绩70%

✓ 作业提交

- 作业提交至“教学云平台”
<https://ucloud.bupt.edu.cn/>

大作业

题目：基于DSL的多业务场景Agent的设计与实现

领域特定语言（DSL）是实现定制化业务流程的核心方法。本作业要求设计一个能够描述特定领域的智能客服机器人的应答逻辑的脚本语言，并实现其解释器。针对来自用户的自然语言，能够调用大型语言模型（LLM）API实现意图识别以驱动脚本的解释执行。本作业旨在评估学生在现代软件开发环境中，融合传统规则引擎与前沿AI技术、进行系统设计、模块化以及严谨验证的能力。

基本要求：

- 脚本语言的语法可以自由定义，只要语义上满足描述客服机器人自动应答逻辑的要求。
- 应该给出几种不同的脚本范例，对不同脚本范例解释器执行之后会有不同的行为表现。
- 选用一种开放API的大语言模型，调用其API对用户的非结构化输入进行意图识别。
- 程序输入输出形式不限，可以简化为纯命令行界面。

大作业

开发环境要求

- 程序设计语言没有限制
- 可以使用类似yacc, lex的工具
- 可以使用大模型辅助开发（如果使用，需要记录开发过程信息在文档中体现，例如，编程过程中，给LLM提的要求，使用的提示词；几轮对话，如果轮数很多，可以简化过程，只说明前几轮和最后几轮的LLM输出结果；如模块划分，接口形式，每个类的定义，需要有明确的要求。LLM的回复是什么，你做了哪些修改。原则是：使用LLM的地方要详细说明使用过程和对LLM输出结果的利用情况。这些内容要体现在文档中）
- 必须使用GIT进行版本管理

大作业

大作业提交的内容：

- 项目全部代码（其中包括测试桩，测试驱动，相关测试所需数据文件，测试脚本）
- PDF文件一份：项目文档，按照文档模板撰写，主要内容包括：
 - 需求分析说明
 - 概要设计说明
 - 详细设计说明（其中应该包括模块之间的接口的详细定义）
 - 测试报告（包括测试桩和测试驱动的设计，自动测试脚本说明，测试结果）
 - 本解释器支持的DSL脚本编写指南（文法定义，用法说明，范例）
 - 如果本项目完成过程中使用了AI辅助编程，需要说明使用的工具，使用的过程描述
 - 完整的GIT日志
- 视频文件一份：程序运行主要功能演示以及讲解录屏
- 答辩PPT

大作业

本作业考察学生规范编写代码、合理设计程序、解决工程问题等方面的综合能力。
满分100分，具体评分标准如下：

- 设计和实现：满分37分，其中数据结构7分，模块划分7分，功能8分，文档15分。
- 接口：满分15分，其中程序间接口8分，人机接口7分。
- 测试：满分23分，测试桩10分，自动测试脚本13分
- 记法：满分10分，文档中对此脚本语言的语法的准确描述。
- 答辩：满分15分，其中PPT 3分，内容陈述9分，回答问题3分。

注意：抄袭或有意被抄袭均为0分

大作业

大作业早答辩加分奖励办法：

- 实验阶段第**1**周答辩：奖励**5**分
- 实验阶段第**2**周答辩：奖励**4**分
- 实验阶段第**3**周答辩：奖励**3**分
- 实验阶段第**4**周答辩：奖励**2**分
- 实验阶段第**5**周答辩：奖励**1**分
- 加分后的实际大作业分数，不超过**100**分

养成使用版本管理工具的习惯



<https://www.w3cschool.cn/git/>

本课程示例代码说明

本课程课件中出现的代码示例，都遵循这个规则：

```
// 错误的代码示例  
int main()  
{  
    retrun 0;  
}
```

浅黄色背景的代码
是错误的，或者有潜在问题的
代码示例

```
// 正确的代码示例  
int main()  
{  
    return 0;  
}
```

浅蓝色背景的代码
是正确的代码示例